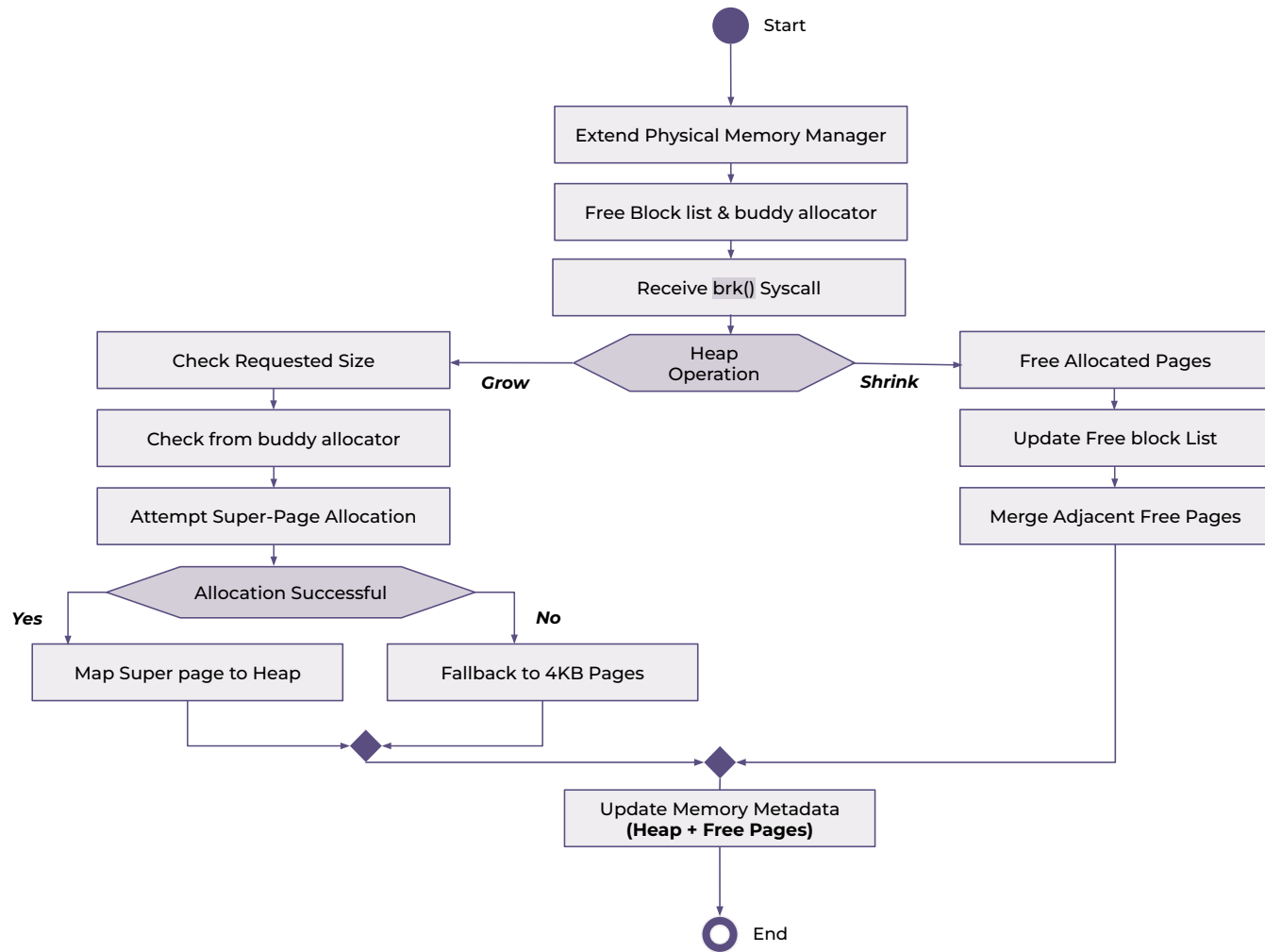

Advanced Memory Allocation

Contiguous and Superpage-Aware Memory Allocation

Group 3
29 | 03 | 2026



Let's move on to



the demonstration!!!!

Objective

1

Extend Physical
Memory Manager
(PMM)

2

structured free
memory block
management

3

Buddy based
contiguous allocator
(2ⁿ order format)

4

Support Superpage
allocation &
deallocation

5

Integrate dynamic
heap operation via `brk`
system call

File-Level Implementation (mCertiKOS)

MATIntro.c

- Core metadata structures
- Free list management
- AT[] operations logic

MATInit.c

- Physical memory detection
- Allocation Table initialization
- Building initial buddy blocks

MATOp.c

- Buddy allocation logic
- Block splitting code
- Block merging implementation

File-Level Implementation (mCertiKOS)

MPTIntro.c

- Low-level page directory and page table code
- Identity mapping and superpage mappings

MPTOp.c

- Virtual address based wrappers
- Address translation logic
- Change mapping using VA

MPTComm.c

- Initializes page directories
- Allocates standard 4KB page tables
- Frees page tables or superpages

File-Level Implementation (mCertiKOS)

MPTKern.c

- Initialize kernel page directory
- Map and unmap virtual pages
- Page table creation & removal

MPTInit.c

- Initial paging environment during system boot
- x86 paging by loading CR3

MPTNew.c

- High-level memory allocation function
- container-based resource accounting

Structure of Allocation Table

This data structure is both:

- Page metadata storage
- $O(1)$ time complexity



```
struct ATStruct {  
    unsigned int perm;  
    unsigned int allocated;  
    unsigned int order;  
    int next;  
    int prev;  
};
```


Memory Zone Division

Calculate split point

```
unsigned int split = VM_USERLO_PI + (VM_USERHI_PI - VM_USERLO_PI) / 4;
```

Segment A

1/4th of the lower memory

Segment B

3/4th of the upper memory

Memory Zone Division (MATInit.c)

- SEGMENT A
- 10 order blocks



```
for i from VM_USERLO_PI to split:
  if page[i] is not usable or already allocated:
    i = i + 1
    continue

  if i is not aligned::
    i = i + 1
    continue

  if block starting at i of size MAX_ORDER-1 is free:
    mark AT[i].order = MAX_ORDER
    add block to free list
    i = i + (1<<(MAX_ORDER-1))
  else:
    i = i + 1
```

Memory Zone Division(MATInit.c)

- SEGMENT B
 - 0-10 order blocks
- 

```
i ← split
While i < VM_USERHI_PI do
  /*check validations*/

  For order = MAX_ORDER-1 down to 0 do
    size ← 2^order
    /*check alignment*/
    /*search for a free block of that order*/
  Endfor

  page[i].order ← order
  add_block_to_freelist(order, i)
  i ← i + 2^order

ENDWHILE
```

Superpage Allocation(MATOp.c)

- First try to allocate in Segment A
 - If fails
Try Segment B
- 

```
if requested order == MAX_ORDER-1 then
```

```
    pindex = find_block_in_zone(order,  
                                SUPER_ZONE_START, SUPER_ZONE_END)
```

```
    if pindex == -1 then
```

```
        pindex = find_block_in_zone(order,  
                                    NORMAL_ZONE_START, NORMAL_ZONE_END)
```

```
    if pindex == -1 then
```

```
        return -1
```

```
    End if
```

Superpage Allocation(MContainer.c)

- Quota check
- Superpage allocation attempt
- Usage update if successful



```
If container_can_consume(id, PAGES_PER_SUPERPAGE)
    pindex = palloc_superpage()
    if pindex != 0 then
        CONTAINER[id].usage = CONTAINER[id].usage +
        PAGES_PER_SUPERPAGE
        return pindex
    else
        Return 0
```

Superpage Allocation(MPTIntro.c)

Set the page
Directory entry



```
value = (page_index << 12) | PT_PERM_PTU | PTE_PS  
PDirPool[proc_index][pde_index] = value
```

Superpage Allocation(MPTComm.c)

Superpage allocation
In VMM layer



```
pindex = container_alloc_superpage(proc_index)
```

```
IF pindex == 0 THEN  
    RETURN 0
```

```
pde_index = vadr >> 22  
pde_entry = (pindex << 12) OR 0x87  
set_pde(proc_index, pde_index, pde_entry)
```

```
RETURN pindex
```

Superpage Deallocation(MATOp.c)

Page deallocation
In PMM layer



```
order = AT[pindex].order
```

```
While order < MAX_ORDER - 1:
```

```
    size = 2^order
```

```
    For i = 0 TO size - 1:
```

```
        at_set_allocated(pindex + i, 0)
```

```
    End for
```

```
    buddy_idx = pindex XOR size
```

```
THEN MERGE BUDDIES IF POSSIBLE
```


Superpage Deallocation(MContainer.c)

Page deallocation
from container



```
pfree_superpage(page_index)  
CONTAINER[id].usage = CONTAINER[id].usage -  
PAGES_PER_SUPERPAGE
```

Superpage Deallocation(MPTComm.c)

Superpage deallocation
In VMM layer



```
if pdir_entry & PTE_PS then
  page_index = pdir_entry >> 12
  rmv_pdir_entry_by_va(proc_index, vadr)

  container_free(proc_index, page_index)

  return
End if
```

Current Implementation Achievements

- Full Physical Memory Initialization & allocation table.
- Greedy Boot-Time Buddy Construction with free list architecture.
- Memory divided into 2 zones to reduce fragmentation.
- Super page allocation & deallocation with container's constraints.

Future Implementations

brk System Call Design

Heap Growth ($\text{new_break} > \text{old_break}$)

- Validate `new_break`
 - Within user space
 - Above heap base
 - Below stack limit
- Align addresses to page boundary
- Compute:
 - `old_page`
 - `new_page`
- For each page in range:
 - Call buddy allocator
 - If allocation fails → rollback previously allocated pages

Heap Shrinking ($\text{new_break} < \text{old_break}$)

- Validate `new_break`
- Align to page boundary
- Compute:
 - `old_page`
 - `new_page`
- For each page being removed:
 - Clear PTE entry
 - Flush TLB
 - Call buddy free function

Free List Architecture

Core Data Structures



Order 10 = 2^{10} = 4MB page
RAM size = 2^{20} = 1048576 pages

```
#define MAX_ORDER 11
#define RAM_PAGES 1048576

struct ATStruct AT[RAM_PAGES];

static int free_list[MAX_ORDER];
```

Initializing Free Lists



```
void pmm_init_freelists(void) {
    for (int i = 0; i < MAX_ORDER; i++) {
        free_list[i] = -1;
    }
}
```

Free List Architecture

Adding a Block to Free List



Head insertion



- New block becomes new head
- Old head becomes next
- Doubly-linked pointers updated

```
void at_list_add(unsigned int order, unsigned int page_index){  
    /*Validate pages and order*/  
  
    int current_head = free_list[order];  
  
    AT[page_index].order = order;  
    AT[page_index].next = current_head;  
    AT[page_index].prev = -1;  
  
    if (current_head != -1) {  
        AT[current_head].prev = page_index;  
    }  
  
    free_list[order] = page_index;  
}
```

Why inserting at head?

Free List Architecture

Removing a Block from Free List

- Get next and previous nodes
- If not head:
 - Update previous node's next
- If head:
 - Move head pointer
- Fix backward pointer of next node
- Clear this node's pointers

```
void at_list_remove(unsigned int order, unsigned int page_index)
{
    /*Validate pages and order*/

    int n = AT[page_index].next;
    int p = AT[page_index].prev;

    if (p != -1)
        AT[p].next = n;
    else
        free_list[order] = n;

    if (n != -1)
        AT[n].prev = p;
}
```

PMM Initialization Strategy

Read memory



Scan memory region



Detect Physical RAM Size



```
devinit(mbi_addr);  
unsigned int n_entries = get_size();  
  
for (i = 0; i < n_entries; i++) {  
    unsigned int end = get_mms(i) + get_mml(i);  
    if (end > highest_addr) highest_addr = end;  
}  
  
unsigned int phys_nps = highest_addr /  
PAGESIZE;
```

PMM Initialization Strategy

Cover user window



Initialize free list



Reset Allocation Table



```
set_nps(VM_USERHI_PI);
```

```
pmm_init_freelists();
```

```
for (i = 0; i < get_nps(); i++) {  
    at_set_allocated(i, 0);  
    at_set_perm(i, 0);  
    AT[i].next = -1;  
    AT[i].prev = -1;  
    AT[i].order = 0;  
}
```

PMM Initialization Strategy

Mark Kernel Pages



```
for (i = 0; i < VM_USERLO_P1; i++) {  
    at_set_perm(i, 1);  
}
```

PMM Initialization Strategy

Classify User Window Pages



Mark as normal page



Mark as reserved



```
for (i = VM_USERLO_PI; i < VM_USERHI_PI; i++){  
  
    unsigned int phys_pi = i - VM_USERLO_PI;  
  
    if (phys_pi >= phys_nps) {  
        at_set_perm(i, 0);  
        continue;  
    }  
  
    /* Check if it lies inside a usable RAM region */  
  
    if (is_ram) {  
        at_set_perm(i, 2);  
        at_set_allocated(i, 0);  
    } else {  
        at_set_perm(i, 0);  
    }  
}
```

Buddy Block Construction

core buddy logic



Try largest possible size



Check alignment and page properties



Alignment formula = $(i \& (size - 1)) == 0$

Skip ahead by full block size



```
i = VM_USERLO_PI;
while (i < VM_USERHI_PI) {

    if (AT[i].perm != 2 || AT[i].allocated != 0) {
        i++;
        continue;
    }

    for (order = MAX_ORDER - 1; order >= 0; order--) {

        if (not_aligned && is_block_free_normal(i, order))
            break;
    }

    AT[i].order = order;
    at_list_add(order, i);

    i += (1U << order);
}
```

Buddy Allocation Procedure

Find Suitable Free Block

Start at requested order

Continue moving upward
Until

- A non-empty free list is found
- Or we reach MAX_ORDER

If none found, allocation failed

Remove Block From Free List

- Store block in pindex

```
unsigned int k = order;
```

```
while (k < MAX_ORDER && get_free_list_head(k) == -1)  
{  
    k++;  
}
```

```
if (k >= MAX_ORDER) {  
    return -1;  
}
```

```
int pindex = get_free_list_head(k);  
at_list_remove(k, pindex);
```

Buddy Allocation Procedure

Split Down to Requested Order

Repeatedly split the block

At each split:

- First half stays in pindex
- Second half becomes a new free block
- That buddy half is added to free_list[k]

```
while (k > order) {  
    k--;  
  
    int buddy = pindex + (1 << k);  
  
    AT[buddy].order = k;  
    at_set_allocated(buddy, 0);  
  
    at_list_add(k, buddy);  
}
```


Buddy Allocation Procedure

Mark Pages as Allocated



Return base address



```
unsigned int size = 1U << order;

for (unsigned int i = 0; i < size; i++) {
    at_set_allocated(pindex + i, 1);
}

AT[pindex].order = order;

return pindex
```

Buddy Deallocation and Merging

Get Block Order →

Mark Entire Block Free →

Compute Buddy Using XOR →

```
unsigned int order = AT[pindex].order;  
if (order >= MAX_ORDER) return;
```

```
unsigned int size = 1U << order;
```

```
for (unsigned int i = 0; i < size; i++) {  
    at_set_allocated(pindex + i, 0);  
}
```

```
unsigned int mask = 1U << order;  
unsigned int buddy_idx = pindex ^ mask;
```

Buddy Deallocation and Merging

Check Merge Conditions

- Higher order must exist
- Buddy inside user window
- Buddy must be free
- Buddy must have same order

```
if (order + 1 < MAX_ORDER &&  
    buddy_idx >= VM_USERLO_PI &&  
    buddy_idx < VM_USERHI_PI &&  
    at_is_allocated(buddy_idx) == 0 &&  
    AT[buddy_idx].order == order)
```

Remove Buddy From Free
List

```
at_list_remove(order, buddy_idx);
```

Buddy Deallocation and Merging

Determine Combined Block Head



```
unsigned int combined;  
if (pindex < buddy_idx) combined = pindex;  
else combined = buddy_idx;
```

Increase Order & Recurse



```
AT[combined].order = order + 1;  
pfree_order(combined);
```

If Merge Not Possible, insert current order



```
else {  
    at_list_add(order, pindex);  
}
```

Extension of Physical Memory Manager

1

Reused original PMM
infrastructure

2

Extended `pmm_init()`
With free list & alignment

3

Integrated buddy system
inside PMM

4

Continued using Allocation
Table (`AT[]`)

How to distinguish between different size of superpages

Free List Architecture

- Array of free lists indexed by order
- Each order represents blocks of size 2^{order}
- Stores only block heads
- Implemented as doubly linked lists
- Enables constant-time insertion and removal

Buddy Allocation Layer

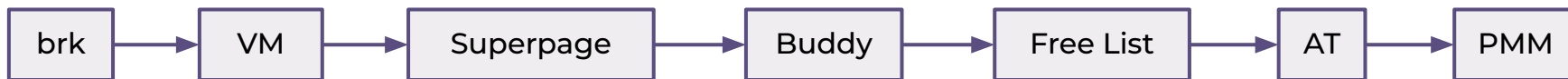
- Organizes memory in power-of-two blocks
- Uses Free Lists to manage blocks
- Supports efficient splitting
- Supports deterministic merging
- Provides contiguous physical memory

Superpage Allocation & Deallocation

- Requests large contiguous blocks from Buddy
- Treats multiple pages as one logical unit
- Handles mapping/unmapping
- Delegates physical allocation to Buddy

brk System Call Integration

- User process requests heap growth/shrink
- brk triggers memory expansion
- Virtual memory layer updates mappings
- Superpage/Buddy provide physical memory



Problem with Existing Memory Allocation

- Only single 4KB page allocation
- No contiguous physical allocation
- No structured free block organization
- No superpage abstraction
- Limited heap growth support

Layered Summary

- PMM → hardware interface
- AT[] → metadata backbone
- Free List → block organization
- Buddy → allocation strategy
- Superpage → large-page abstraction
- brk → user-level interface

Integration with Existing mCertiKOS

- **Modules Extended:**

- `pmem_init()` → Memory initialization
- Physical Memory Manager (PMM)
- System call handling

- **Structures Reused:**

- `AT[]`
- Page tables
- Virtual memory infrastructure

Integration with Existing mCertiKOS

- **What We Did NOT Modify:**

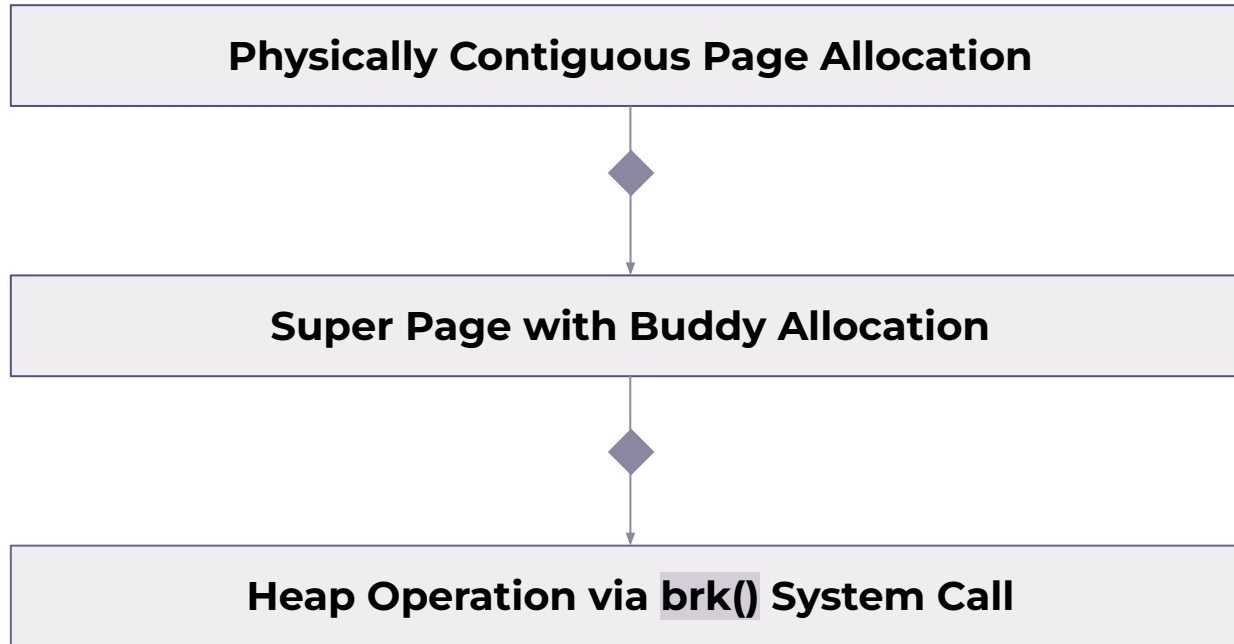
- Kernel space handling
- Scheduler
- Trap handling

Trade-offs

Alignment constraints

No Arbitrary size of
superpage

Overview

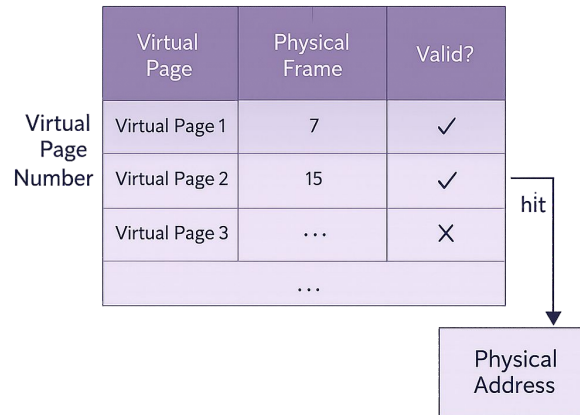


Motivation

- Single-page allocation is inefficient for large memory requests
- DMA-based applications require physically contiguous memory

Superpages reduce:

1. TLB Misses
2. Page table lookup overhead



Core Problem Statement

The existing memory allocation mechanism in mCertiKOS is limited to single 4KB page allocation and lacks support for physically contiguous pages, super-page mappings, and efficient memory deallocation.

Objective

1

**Implement Physically
Contiguous Page
Allocator**

2

**Enable Super Page
Allocation using x86
PSE**

3

**Support Page
Deallocation Using
brk() System Call**

4

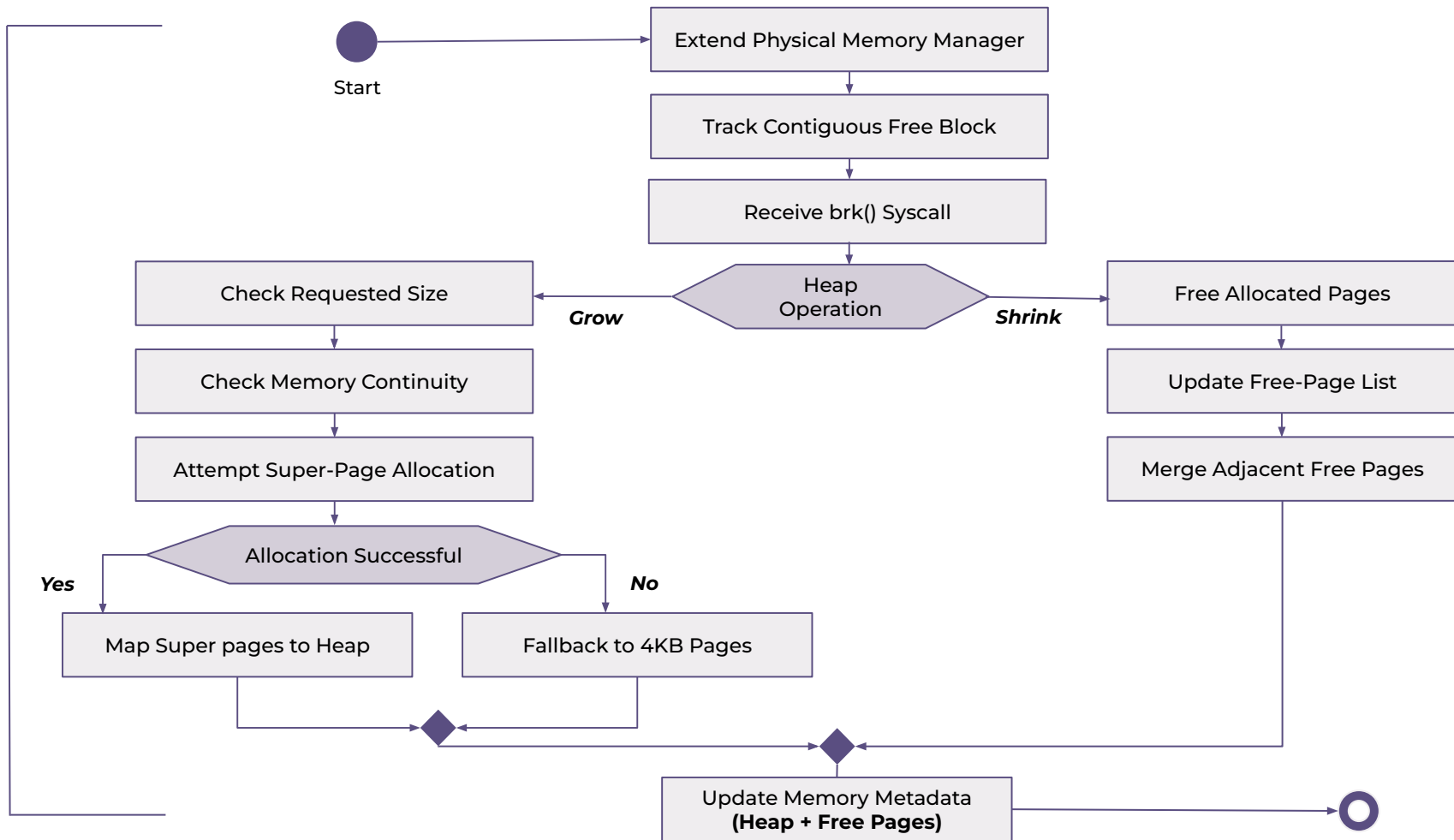
**Handle Page
Fragmentation**

5

**Reducing TLB misses
and Page Table
Overhead**

Super Page

Proposed Approach (Flow Chart)



Proposed Approach(Big steps)

- **Extend physical memory manager to track contiguous free pages**
- **Modify `brk()` syscall to:**
 - Grow heap by allocating contiguous pages
 - Shrink heap by freeing pages
- **Attempt heap operation with buddy allocation:**
 - If heap growing → super page/ normal page allocation
 - If heap shrinking → page deallocation and merging
- **Update Heap Metadata**

Buddy Allocation

Proposed Functions

- **buddy_alloc(order)**

- Allocates 2^{order} physically contiguous pages
- Split larger blocks if needed
- Mark the blocks as allocated

- **buddy_alloc_consecutive(num_pages)**

- Find smallest order such that $2^{\text{order}} \geq \text{num_pages}$
- Allocate a contiguous block using **buddy_alloc(order)**

Proposed Functions

- **buddy_free(page_index, order)**

- Frees a block of size 2^{order}
- Merges with buddy if possible

- **sys_brk(proc_index, vaddr, size, flags)**

- **BRK_SUPERPAGE**
 - Call `superpage_alloc()`
- **BRK_CONSECUTIVE**
 - Allocate contiguous pages via buddy allocator
 - Map pages using normal PTEs
- **Default mode**
 - Allocate pages individually
 - Use standard page tables

Proposed Functions

- **superpage_alloc()**

- Allocate a 2MB physically contiguous block
- Return base physical page index

- **superpage_free(page_index)**

- Free the 2MB block back to buddy allocator
- Restore it as a normal free block

Proposed Algorithm

buddy_alloc():

receive a memory allocation request
search for the smallest free block that can satisfy
the request

if a suitable block is found:

allocate the block and mark it as used
return the starting address

if only a larger block is found:

split the larger block into two smaller blocks
keep splitting until the requested size is
matched
allocate the required block
return the starting address

if no block is available:

report allocation failure

buddy_init():

*/*necessary initialization in user space*/*

buddy_free():

receive a memory free request
mark the memory block as free
check whether the neighboring buddy block is
also free

if the buddy block is free:

merge both blocks into a larger continuous
block
repeat merging while possible

insert the final merged block into the free
memory list

Relevant Operating System Concepts

Paging & Virtual
Memory

Page Size Extension (PSE)

Page Tables and Page
Directories

Memory Fragmentation
(physical-level)

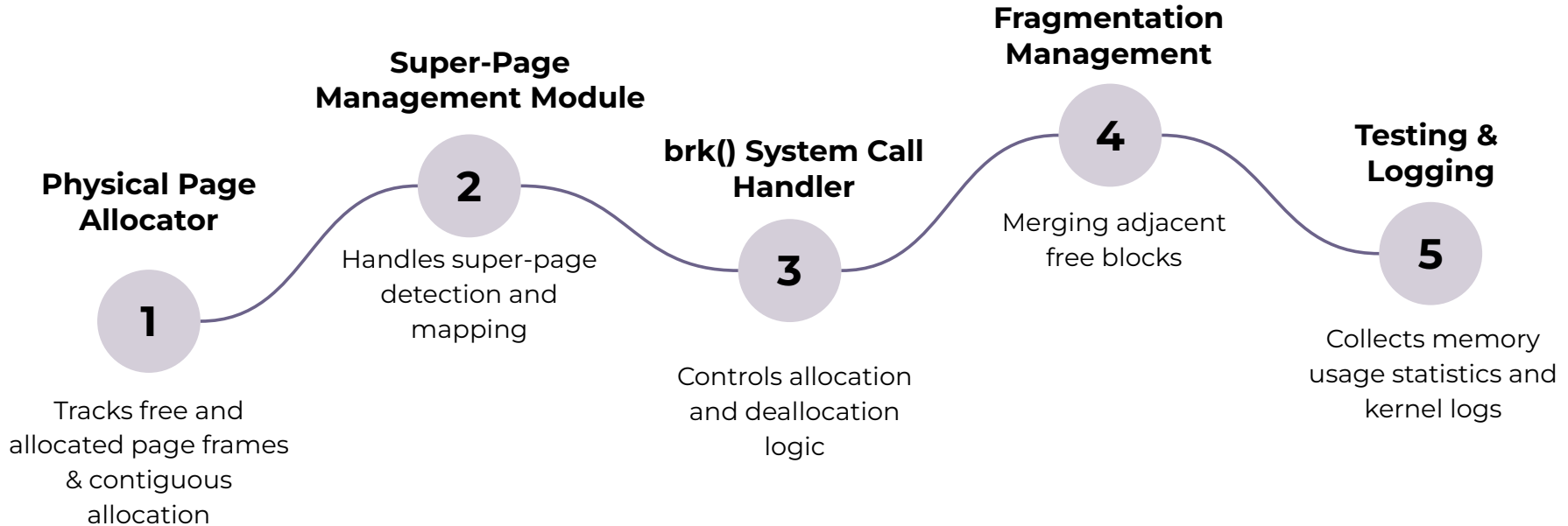
Dynamic Memory
Allocation

System calls (brk())

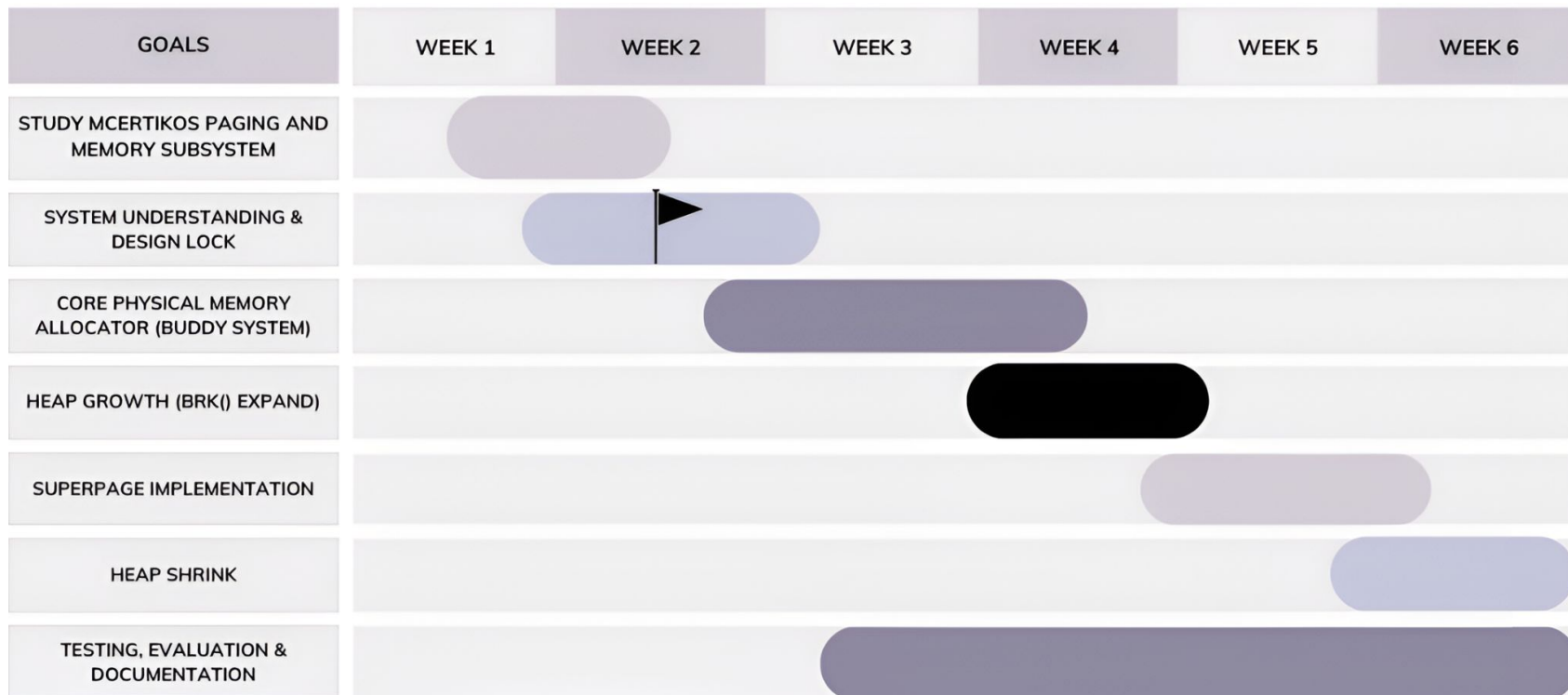
Buddy Allocation

Translation Lookaside
Buffer

Major Components



Timeline with Key Milestones



Division of Work Among Team Members

A vertical line on the left side of the slide, with five colored circles (light purple, medium purple, dark purple, dark purple, dark purple) corresponding to each team member's section.

Israt Risha Ivey - 220042103

- Buddy Allocator Core , Integration and Testing

Ramisa Anan Rahman - 220042105

- Buddy Allocator Core, Integration and Testing

Ridika Naznin - 220042115

- Heap Growth (brk() expand), Integration and Testing

Zannatul Adon Sabiha- 220042123

- Superpage Support, Integration and Testing

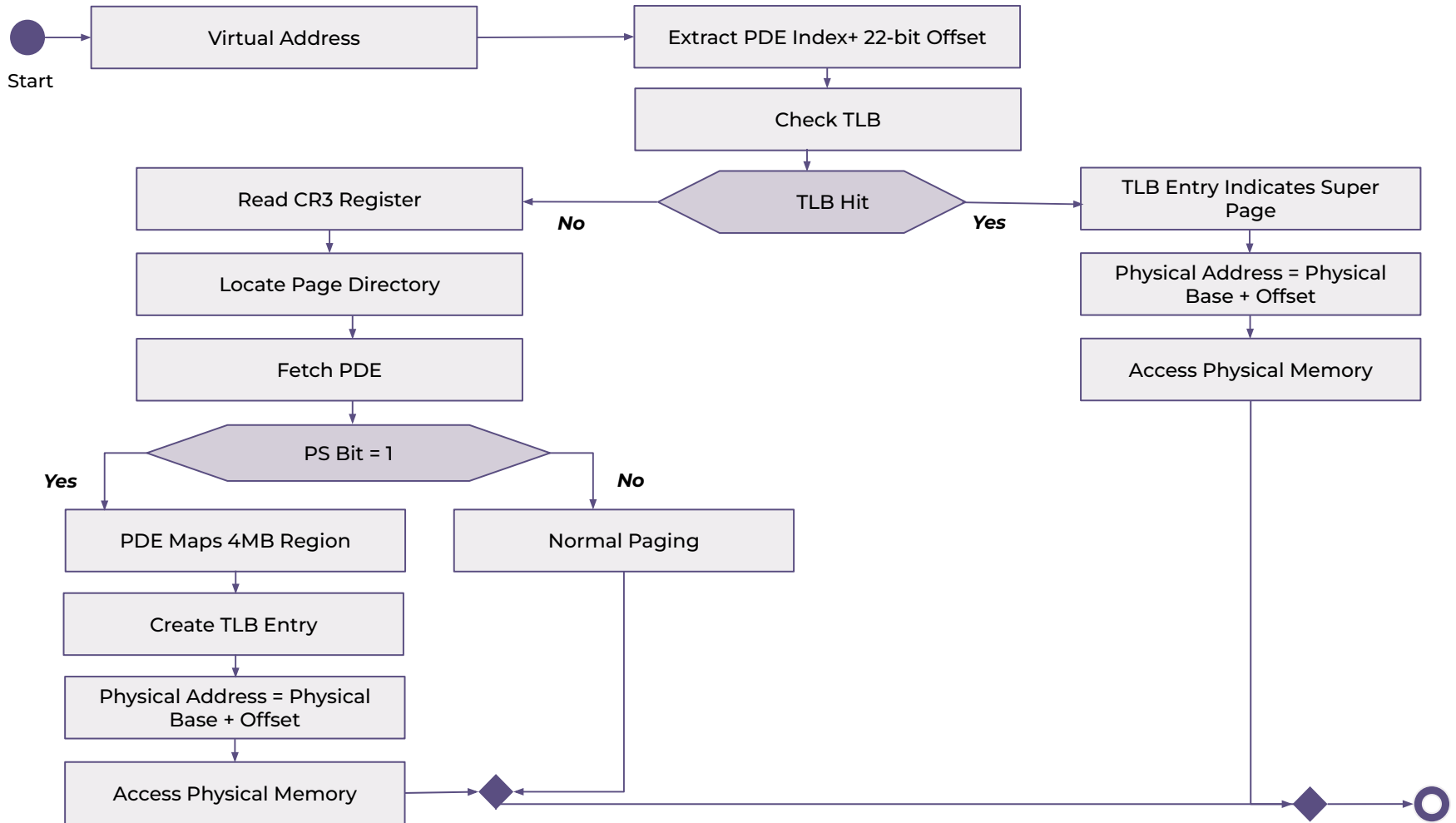
Jarin Subha Sneha - 220042161

- Heap Shrink & Deallocation, Integration and Testing

Thank You for your Attention!!!!

We Are Open To Questions...

TLB For Super Page



Major Components

Physical Page Allocator	Tracks free and allocated page frames & contiguous allocation
Super-Page Management Module	Handles super-page detection and mapping
brk() System Call Handler	Controls allocation and deallocation logic
Fragmentation Management	Merging adjacent free memory regions
Testing & Logging	Collects memory usage statistics and kernel logs

High-Level Architecture (mCertiKOS)

Timeline with Key Milestones

